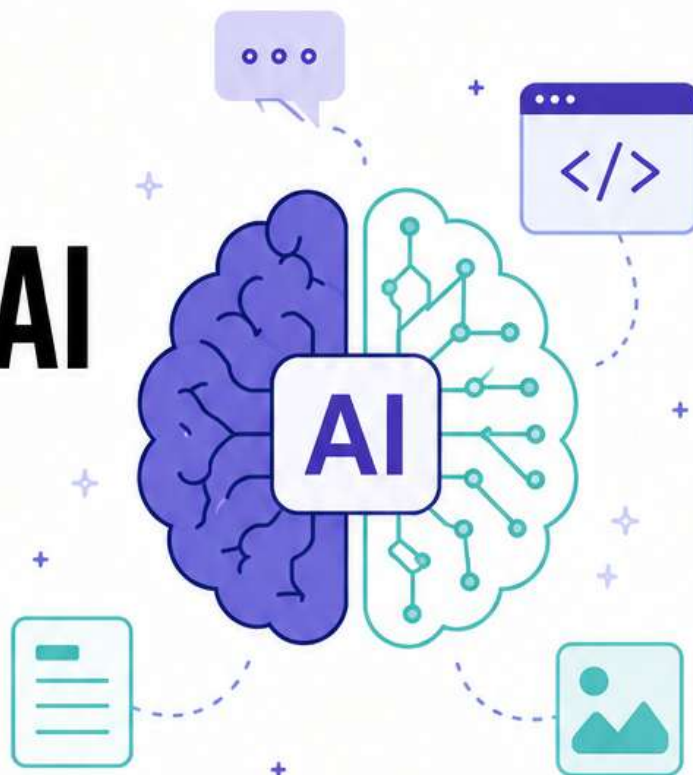




20 GENERATIVE AI INTERVIEW QUESTIONS

Medium & Hard Questions
with Concise Answers



COVERING THE MOST IMPORTANT TOPICS



LLMs &
Transformers



Prompt
Engineering



Embeddings &
Vector Databases



RAG
(Retrieval-Augmented
Generation)



Fine-tuning vs
RAG vs Prompt
Engineering



AI Agents &
Tool Calling



Inference &
Optimization



Evaluation &
Hallucinations



Advanced Concepts
& System Design



Best Practices
& Tips



WHAT YOU'LL GET

- ✓ 20 carefully selected interview questions
- ✓ Concise, crisp and to-the-point answers
- ✓ Diagrams & visuals to understand better
- ✓ Covers medium to hard level questions
- ✓ Perfect for interviews & real-world roles



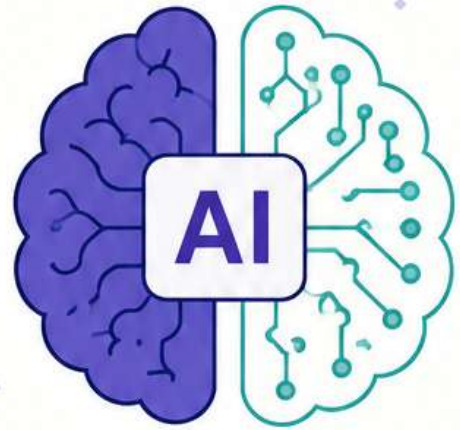
INTERVIEW TIP

Interviewers don't just test what you know—they test how well you think, explain and apply concepts to real-world problems.

**Understand deeply. Think clearly.
Answer confidently.**



LLMs & TRANSFORMERS



Foundation concepts you must know for every AI role.

01 What is a Large Language Model (LLM)?

- An LLM is an AI model trained on massive text data to understand and generate human-like text based on patterns.



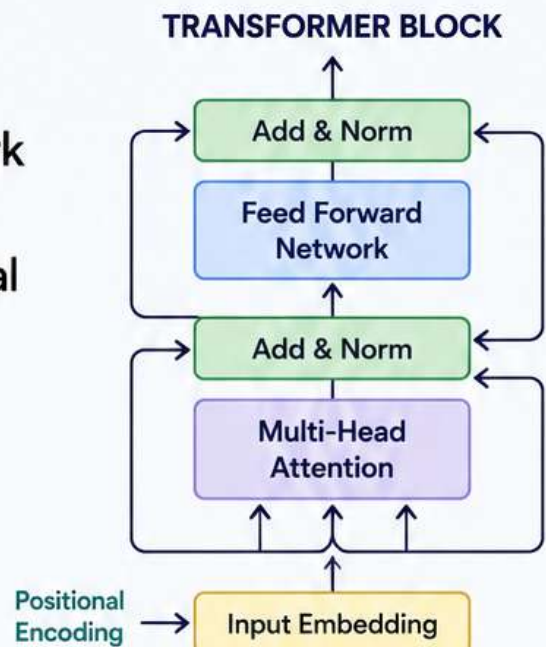
02 What is a Transformer?

- A Transformer is a neural network architecture that uses attention mechanisms to process sequential data efficiently in parallel.

HOW ATTENTION WORKS



The model focuses on relevant words and gives more importance to them.



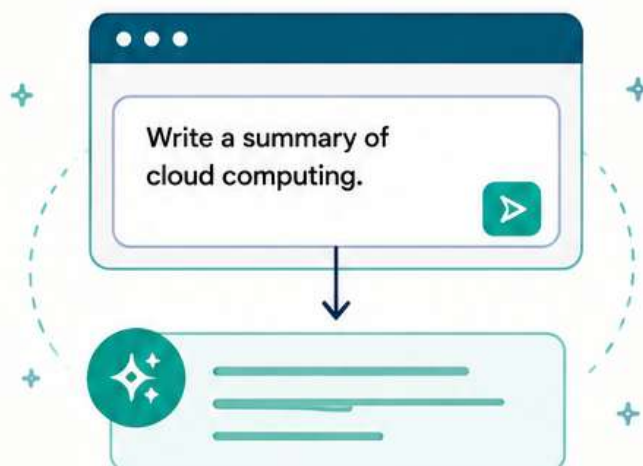
INTERVIEW TIP

Understand how attention helps the model capture context and relationships between words.



PROMPT ENGINEERING

Good prompts get better answers from AI.



01 What is Prompt Engineering?

- Prompt Engineering is the process of designing and refining inputs (prompts) to guide AI models toward better, more accurate and relevant outputs.



02 Why is Prompt Engineering Important?

- It improves output relevance, reduces hallucinations, saves tokens & cost, and helps models understand your intent clearly.



ANATOMY OF A GOOD PROMPT



ROLE

Define the role for the model.

e.g., You are a helpful data analyst.

+



TASK

Clearly state what you want the model to do.

e.g., Summarize the following article.

+



CONTEXT

Provide necessary background information or data.

e.g., The article is about Generative AI trends in 2024.

+



CONSTRAINTS

Add rules or limitations for better control.

e.g., In 3 bullet points. Simple language.

+



FORMAT

Specify the output format you expect.

e.g., Bullet points in markdown format.



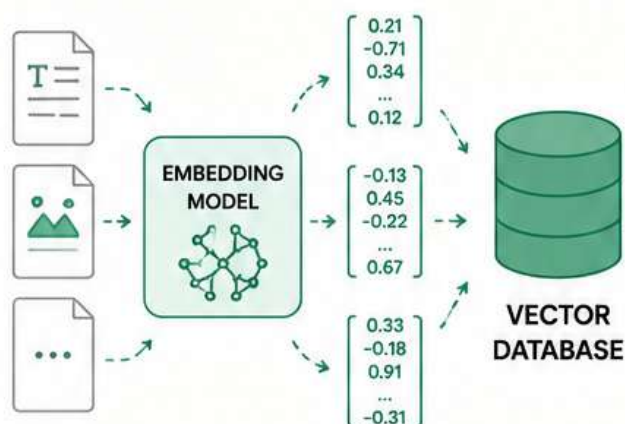
INTERVIEW TIP

Clear intent + relevant context + proper format
= better results every time.



EMBEDDINGS & VECTOR DATABASES

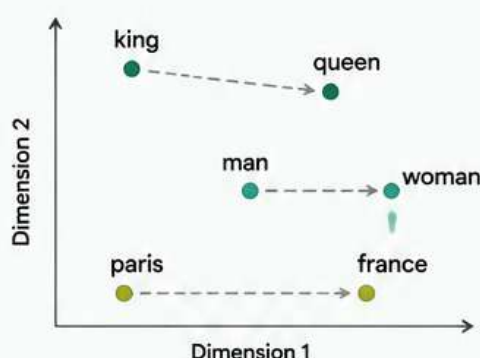
The bridge between unstructured data and intelligent retrieval.



01 What are Embeddings?

- Embeddings are numerical representations of text, images or other data in high-dimensional space that capture their meaning and relationships.

Words in Vector Space (2D Example)



02 What is a Vector Database?

- A Vector Database is a specialized database designed to store, index and retrieve high-dimensional embeddings efficiently using similarity search.



Similarity Search

Finds items most similar to a query vector.



Scalable Storage

Handles millions to billions of vectors.



Fast Retrieval

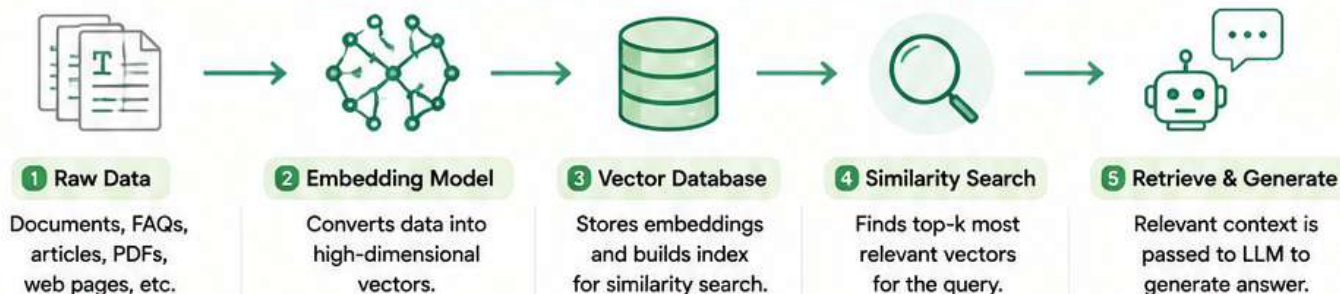
Optimized indexes for low latency search.



Built for AI Workloads

Powering RAG, recommendations, semantic search and more.

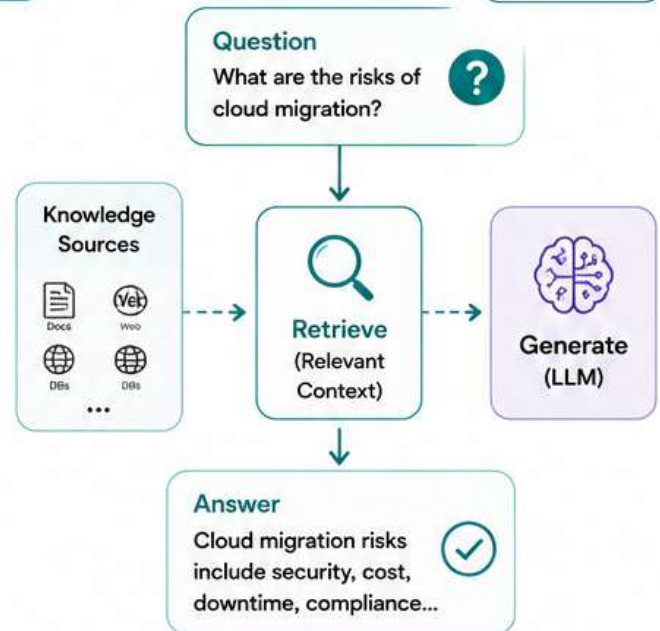
• EMBEDDING → VECTOR DB → RETRIEVAL PIPELINE •



INTERVIEW TIP

Embeddings capture meaning. Vector DBs make it searchable. Together, they power intelligent AI systems.

Combine the power of knowledge retrieval
with generative models.



01 What is RAG?

- RAG is an approach that retrieves relevant information from external knowledge sources and provides it to the LLM as context before generating the final answer.

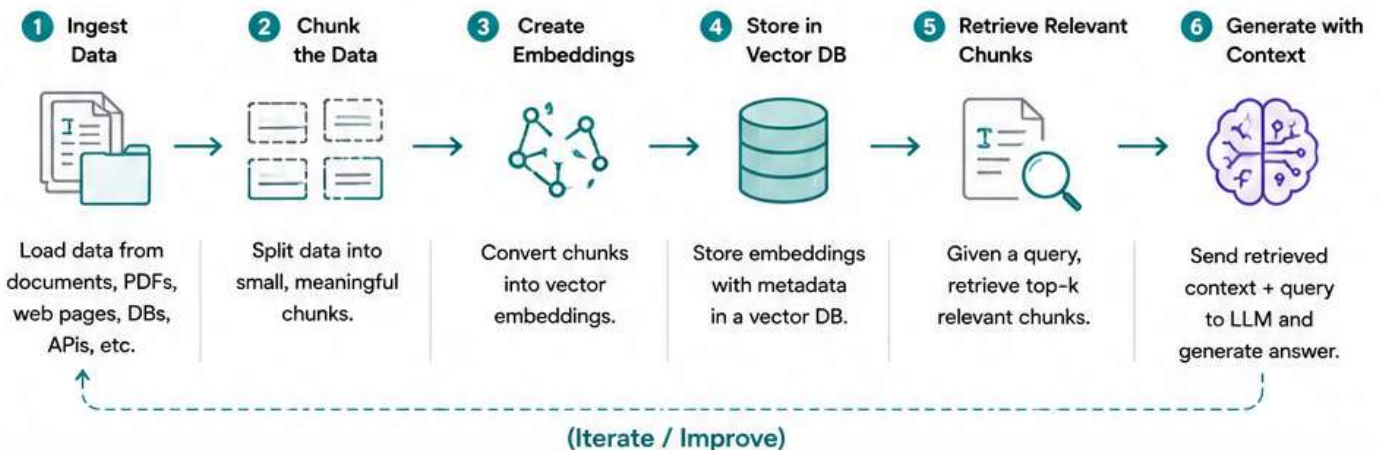


02 Why use RAG?

- Access up-to-date and domain-specific knowledge
- Reduce hallucinations and improve accuracy
- Provide explainable and verifiable answers
- No need to fine-tune the model



THE RAG PIPELINE

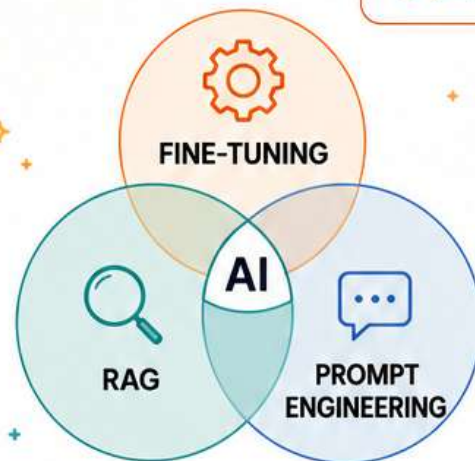


INTERVIEW TIP

**RAG = Retrieve relevant knowledge + Augment the prompt
+ Generate better, grounded answers.**



FINE-TUNING VS RAG VS PROMPT ENGINEERING



Three powerful approaches.
Choose the right one for the right problem.

01 What is Fine-tuning?

- Fine-tuning is training a pre-trained model on a specific dataset to adapt it to a particular task or domain.



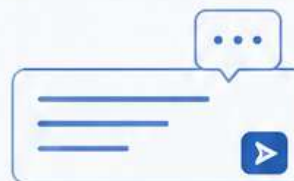
02 What is RAG?

- RAG retrieves relevant information from external sources and provides it to the LLM to generate accurate and up-to-date answers.



03 What is Prompt Engineering?

- Prompt Engineering is designing and refining inputs (prompts) to guide the LLM to produce better, more accurate and consistent outputs.



COMPARISON AT A GLANCE

ASPECT	FINE-TUNING	RAG	PROMPT ENGINEERING
What it is	Train the model on your own data.	Retrieve external knowledge at inference time.	Optimize the input instructions (good prompts).
Best for	Specialized tasks, specific domain behavior.	Up-to-date info, large knowledge bases, accuracy.	General tasks, quick iteration, improve output quality.
Data required	Large, labeled, high-quality dataset.	Curated documents/ knowledge sources.	No extra training data required.
Cost	High (training is expensive).	Medium (infra + vector DB + retrieval).	Low (just prompt design and iteration).
Implementation complexity	High (training pipeline required).	Medium (chunking, embeddings, vector DB, retrieval).	Low (craft, test and iterate prompts).
Update knowledge	Requires retraining.	Just update the knowledge base.	No new knowledge added.
Control	High control over model behavior.	Medium control (depends on retrieval quality).	Medium control through prompt design.
Latency	Low (after training).	Higher (retrieval + generation).	Low.
Risk of Hallucinations	Medium (model may still hallucinate).	Low (grounded in retrieved context).	High to Medium (depends on prompt quality).



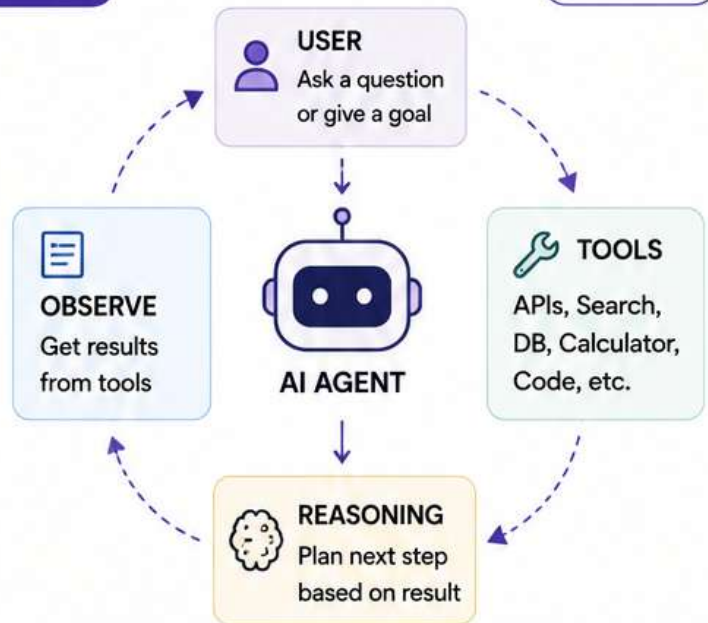
INTERVIEW TIP

There is no one-size-fits-all. Understand the problem, data availability, latency needs and choose the right approach or a combination.



AI AGENTS & TOOL CALLING

Agents take actions. Tools give them real-world capabilities.



01 What is an AI Agent?

- An AI Agent is an autonomous system that perceives its environment, reasons about goals, and takes actions using tools to achieve those goals.

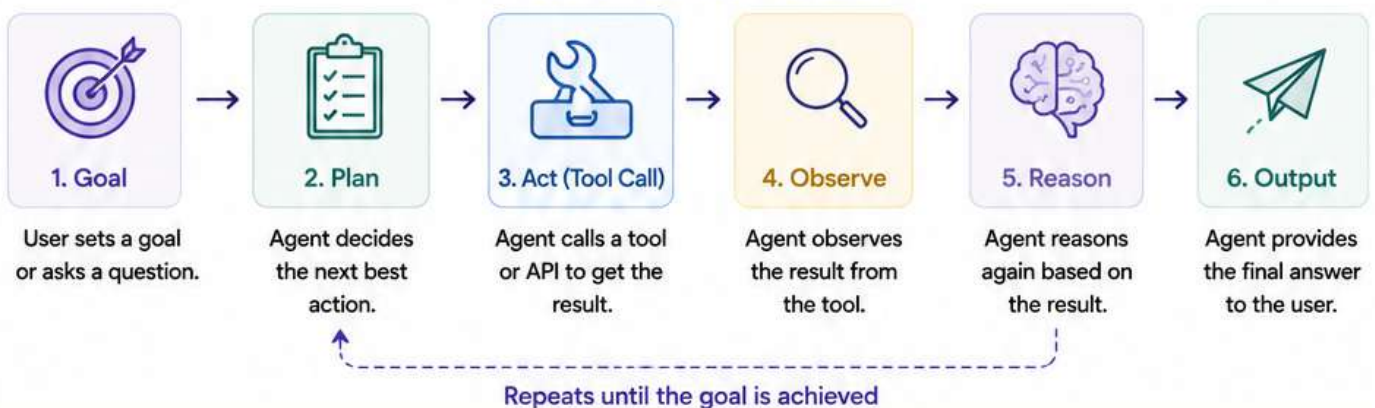


02 What is Tool Calling?

- Tool Calling allows an LLM to invoke external functions or APIs to fetch information, perform actions or manipulate data.



• HOW AN AGENT WORKS •



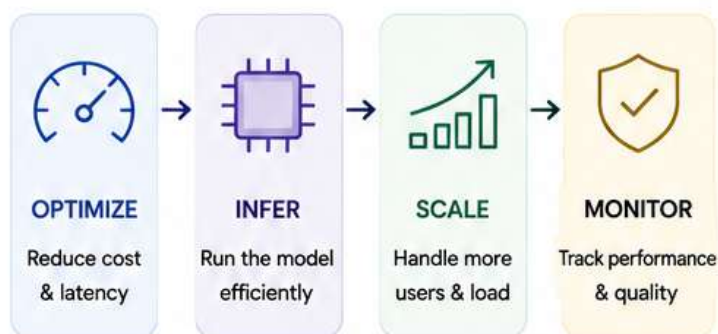
INTERVIEW TIP

Think of an agent as an LLM with a brain (reasoning), eyes (observation) and hands (tools).



OPTIMIZATION & INFERENCE

Make models faster, cheaper and production-ready.



01 What is Optimization?

- Optimization techniques improve model performance, reduce latency, lower costs and make models efficient for real-world deployment.



02 What is Inference?

- Inference is the process of using a trained model to make predictions or generate outputs on new, unseen data.

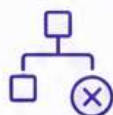


03 Key Optimization Techniques



Quantization

Reduce precision (e.g., from FP32 to INT8) to lower memory and speed up inference.



Pruning

Remove less important weights or neurons to make the model smaller and faster.



Knowledge Distillation

Train a smaller model (student) to mimic a larger model (teacher) while maintaining performance.



Model Compression

Compress model size using techniques like weight sharing, low-rank factorization, etc.



Caching

Cache frequent requests/responses to reduce latency and cost.

04 Best Practices for Inference



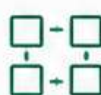
Choose the Right Model Size

Balance accuracy, latency and cost based on use case.



Use Efficient Hardware

Leverage GPUs, TPUs or specialized accelerators.



Batch Requests

Process multiple inputs together to improve throughput.



Monitor Latency & Throughput

Continuously track performance and identify bottlenecks.



A/B Test & Iterate

Continuously test different models and configurations to improve results.

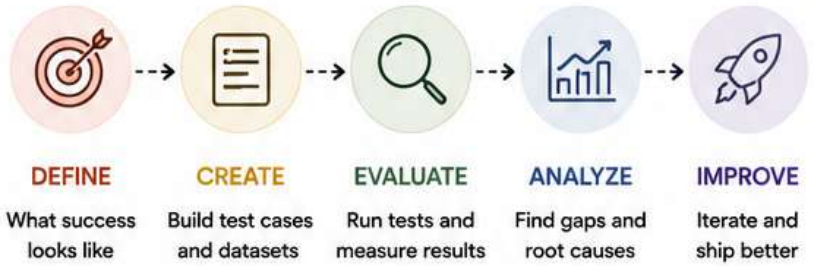


INTERVIEW TIP

Optimization is about efficiency. Inference is about execution. Together, they deliver great user experiences at scale.



EVALUATION & TESTING



Measure what matters.

Build reliable and trustworthy AI.

01 Why Evaluate?

- Ensure accuracy, reliability and safety
- Catch issues early and reduce risk
- Improve continuously with data
- Build user trust and business impact



02 What to Evaluate?

- Correctness (is it right?)
- Relevance (is it helpful?)
- Safety (is it safe & aligned?)
- Robustness (does it handle edge cases?)
- Performance (latency, cost, throughput)



03 Key Evaluation Metrics



Accuracy

How often the output is correct.

Example: Exact match, F1 score



Relevance

How well the output addresses the query.

Example: Relevance score (0-1)



Safety

Checks for harmful, biased or unsafe content.

Example: Safety classifier score



Robustness

How well it performs on edge cases, noise and unexpected inputs.

Example: Pass@K, Error rate



Performance

Speed, cost and resource efficiency.

Example: Latency (ms), Cost per 1K tokens

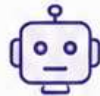
04 Evaluation Approaches



Human Evaluation

Experts review outputs for quality, nuance and subjective criteria.

Best for: Quality, creativity, tone



LLM-as-a-Judge

Use another LLM to score or rank outputs based on rubrics.

Best for: Scalable, consistent scoring



Automated Metrics

Use algorithms and benchmarks to measure outputs quantitatively.

Best for: Accuracy, factuality, speed



A/B Testing

Compare model versions with real users to see what works better.

Best for: Real-world impact and adoption

05 Best Practices



Define clear goals and success metrics



Use diverse and realistic test data



Evaluate continuously (not just once)



Track trends and monitor in production



Involve humans in the loop



INTERVIEW TIP

Good evaluation isn't about perfection.

It's about measuring the right things and improving every day.



EVALUATION & BEST PRACTICES

Build reliable, safe and effective AI systems.



EVALUATION FRAMEWORKS



1. Correctness

- Is the answer factually correct?
- Use golden sets, unit tests, or exact match.



2. Relevance

- Is the response relevant to the query?
- Measure context precision.



3. Faithfulness

- Is the answer supported by the provided context?
- Avoids hallucinations.



4. Coherence

- Is the response clear, complete and well-structured?
- Measure readability and flow.



5. Task Success

- Does the output solve the user's goal?
- End-to-end success metric.

COMMON EVAL METRICS



Accuracy



Precision@k



Recall@k



F1 Score



Latency



User Rating

BEST PRACTICES



1. Define Clear Goals

Start with a specific use case and success criteria.



2. Provide Quality Context

Clean, relevant and well-structured data improves results.



3. Design Good Prompts

Be specific, provide examples and define the output format.



4. Use the Right Tools

Connect reliable APIs and data sources your agent can trust.



5. Add Guardrails

Implement filters, validation, rate limits and permission checks.



6. Monitor & Iterate

Track performance, collect feedback and continuously improve.



INTERVIEW TIP

Great AI systems are not just built — they are evaluated, secured and improved continuously.